第20章 UCOSII操作系统

20.1 UCOSII简介

- UCOSII 的前身是 UCOS，最早出自于 1992 年美国嵌入式系统专家 Jean J.Labrosse 在《嵌入式系统编程》杂志的 5 月和 6 月刊上刊登的文章连载，并把 UCOS 的源码发布在该杂志的BBS 上。
- 目前最新版本：UCOSIII，但是现在使用最为广泛的还是 UCOSII
- UCOSII 是一个可以基于 ROM 运行的、可裁减的、抢占式、实时多任务内核，具有高度可移植性，特别适合于微处理器和控制器，是和很多商业操作系统性能相当的实时操作系统(RTOS)。

20.1 UCOSII简介（续）

- 为了提供最好的移植性能，UCOSII 最大程度上使用 ANSI C 语言进行开发，并且已经移植到了几乎所有知名的 CPU 上，涵盖了从 8 位到 64 位各种 CPU(包括 DSP)。

- UCOSII 是专门为计算机的嵌入式应用设计的， 绝大部分代码是用 C 语言编写的。CPU 硬件相关部分是用汇编语言编写的、总量约 200 行的汇编语言部分被压缩到最低限度，为的是便于移植到任何一种其它的 CPU 上。

20.1 UCOSII简介（续）

- 用户只要有标准的 ANSI 的 C 交叉编译器，有汇编器、连接器等软件工具，就可以将 UCOSII 嵌入到开发的产品中。

- UCOSII 特点：执行效率高、占用空间小、实时性能优良和可扩展性强， 最小内核可编译至 2KB 。

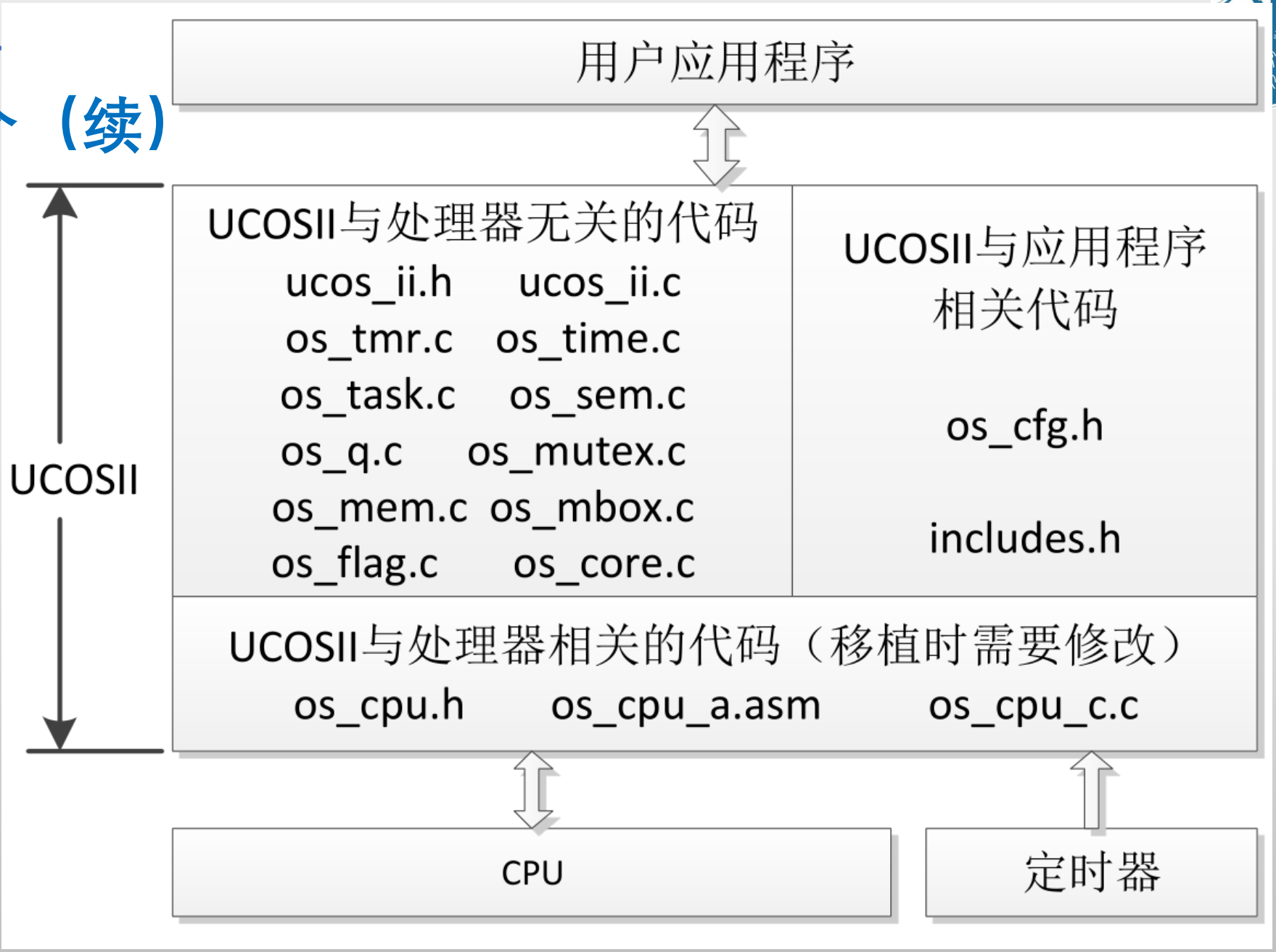
- UCOSII 构思巧妙。结构简洁精练，可读性强，同时又具备了实时操作系统的全部功能，非常适合RTOS初学者

20.1 UCOSII简介（续）

- 应用领域
  - 教育机器人系统
  - 数码相框系统应用
  - 小型PLC控制系统
  - 工业控制领域应用
  - 智能视觉监控系统
  - 船载导航系统应用研究

20.1 UCOSII简介（续）

- **UCOSII体系结构图**



- **UCOSII 的移植，只需要修改：os_cpu.h、os_cpu_a.asm 和 os_cpu.c三个文件即可**
- os_cpu.h：进行数据类型的定义，以及处理器相关代码和几个函数原型；
- os_cpu_a.asm：移植过程中需要汇编完成的一些函数，主要是任务切换函数；
- os_cpu.c：定义一些用户 HOOK 函数。

➢ 附：HOOK 函数（钩子函数）

- 钩子函数（Hook Function）是一种在软件开发中用于监视和修改系统消息的技术。它允许程序截获原本发往其他程序或窗口的消息，并对这些消息进行处理、修改或传递。
- Hook 函数在UCOSII中的作用：能够通过系统调用实现对进程或系统行为的监控和修改。

① **消息处理**：钩子函数可以用来截获并处理特定的消息。在UCOSII中，这意味着开发者可以定义钩子函数来响应特定的事件或消息，从而实现更灵活的控制和数据处理机制。

② **系统调试和分析**：钩子函数还可以用于系统调试，例如通过记录和回放日志来帮助开发者理解程序的运行状态和性能问题。这在使用WHJOURNALRECORD和WHJOURNALPLAYBACK这样的钩子函数时尤为重要，它们可以帮助开发者追踪和分析程序的行为。

③ **用户界面改进**：在UCOSII中，钩子函数也可以用于增强用户界面。例如，可以通过设置快捷键来快速显示或隐藏应用程序，或者修改对话框的功能和外观，以提高用户交互的效率和友好性。

④ **窗口管理**：钩子函数还可以用于窗口管理，如通过动态链接库插入来实现窗口的子类化，从而改变窗口的行为特性。这在需要高度定制化窗口行为的应用程序开发中非常有用。

- 定时器作用：为 UCOSII 提供系统时钟节拍，实现任务切换和任务延时等功能。
  - 这个时钟节拍由 OS_TICKS_PER_SEC（在 os_cfg.h 中定义）设置，一般时钟节拍设置为 1ms~100ms。
  - 本章，利用 STM32F4的 SYSTICK 定时器来提供 UCOSII 时钟

20.1 UCOSII简介（续）

- UCOSII 早期版本只支持 64 个任务，但是从 2.80 版本开始，支持任务数提高到 255 个。

- UCOSII 保留了最高4 个优先级和最低 4 个优先级的总共 8 个任务，用于拓展使用，但实际上，UCOSII 一般只占用了最低 2 个优先级，分别用于空闲任务（倒数第一）和统计任务（倒数第二），所以剩下给用户使用的任务最多可达：

  255-2=253 个（V2.91）。

20.2 UCOSII任务(Task)

- **任务**：所谓任务，其实就是一个死循环函数，该函数实现一定的功能。

- 一个工程可以有很多这样的任务（最多 255 个），UCOSII 对这些任务进行调度管理，让这些任务可以并发工作（注意不是同时工作！！，并发只是各任务轮流占用 CPU，而不是同时占用，任何时候还是只有 1个任务能够占用 CPU），这就是 UCOSII 最基本的功能。

- UCOS中，所有任务（除空闲任务外）必须在合适的位置调用系统延时函数或事件等待函数，以保证低优先级的任务得到CPU控制权。

20.2 UCOSII任务（续）

- UCOS 任务的一般格式：

```
void MyTask (void *pdata)
{
    任务准备工作…
    While(1)//死循环
    {
        任务 MyTask 实体代码;
        OSTimeDlyHMSM(x,x,x,x);//调用任务延时函数，释放 cpu 控制权，
    }
}
```

20.2 UCOSII任务（续）

➢ UCOS 任务调度方法：

假如同时运行 2 个任务： MyTask 和 YourTask, 先忽略任务优先级的概念，两个任务死循环中延时时间为 1s。

如果某个时刻，任务 MyTask 在执行中，当它执行到延时函数OSTimeDlyHMSM的时候，它释放 cpu 控制权，这个时候，任务 YourTask 获得 cpu 控制权开始执行，任务 YourTask 执行过程中，也会调用延时函数延时 1s 释放 CPU 控制权，这个过程中任务 A 延时 1s 到达，重新获得 CPU 控制权，重新开始执行死循环中的任务实体代码。

如此循环，现象就是两个任务交替运行，就好像 CPU 在同时做两件事情

20.2 UCOSII任务（续）

➢ UCOS 任务调度方法：

如果有很多任务都在等待，那么先执行那个任务呢？如果任务在执行过程中，想停止之后去执行其他任务是否可行呢？（任哲：《UCOSII 实时操作系统》）

比如：音乐播放器实验，在音乐播放的时候，我们还希望显示歌词，如果是 1 个死循环（一个任务），那么很可能在显示歌词的时候，音频可能出现停顿（尤其是采样率高的时候），这主要是歌词显示占用太长时间，导致 IIS 数据无法及时填充而停顿。

解决方案：可以分 2 个任务，音乐播放任务（优先级高），歌词显示任务（优先级低）。这样，由于音乐播放任务的优先级高于歌词显示任务，音乐播放任务可以打断歌词显示任务，从而及时给 IIS 填充数据，保证音频不断，而显示歌词又能顺利进行。这就是 UCOSII 带来的好处。

20.2 UCOSII任务（续）

- **任务优先级：**

  数值越小，优先级越高

  任务就绪表中总是优先级最高的任务获得 CPU 使用权，只有高优先级的任务让出 CPU 使用权（比如延时）时，低优先级的任务才能获得 CPU 使用权

  UCOSII 不支持多个任务优先级相同，也就是每个任务的优先级必须不一样。

- **任务堆栈：**存储器中的连续存储空间。

  为了满足任务切换和响应中断时保存 CPU 寄存器中的内容以及任务调用其他函数时的需要，每个任务都有自己的堆栈。

  任务堆栈是任务创建的一个重要入口参数。

20.2 UCOSII任务（续）

- **任务控制块OS_TCB：**

用来记录任务堆栈指针，任务当前状态以及任务优先级等任务属性。

UCOSII 的任何任务都是通过OS_TCB的东西来控制，一旦任务创建了，OS_TCB就会被赋值。

每个OS_TCB有 3 个最重要的参数：

1，任务函数指针；

2，任务堆栈指针；

3，任务优先级；

任务控制块就是任务在系统里面的身份证（UCOSII 通过优先级识别任务)

20.2 UCOSII任务（续）

- **任务就绪表：**

用来记录系统中所有处于就绪状态的任务。

是一个位图，系统中每个任务都在这个位图中占据一个进制位，该位置的状态（1 或者 0）就表示任务是否处于就绪状态。

- **任务调度：**

任务调度作用：

1. 在任务就绪表中查找优先级最高的就绪任务
2. 实现任务的切换。（见P12举例）

20.2 UCOSII任务（续）

- **任务状态：**

睡眠状态，任务在没有被配备任务控制块或被剥夺了任务控制块时的状态。

就绪状态，系统为任务配备了任务控制块且在任务就绪表中进行了就绪登记，任务已经准备好了，但由于该任务的优先级比正在运行的任务的优先级低， 还暂时不能运行，这时任务的状态叫做就绪状态。

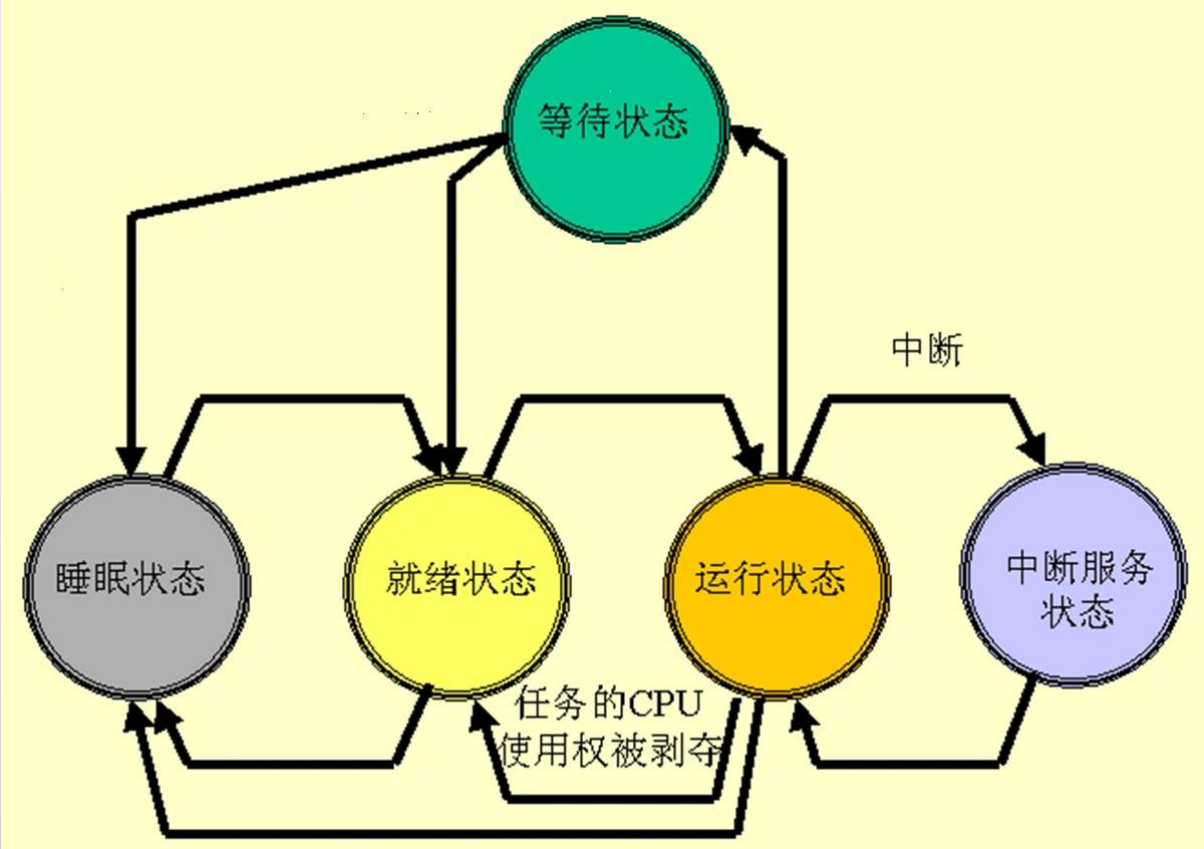
运行状态，该任务获得 CPU 使用权，并正在运行中，此时的任务状态叫做运行状态。

等待状态，正在运行的任务，需要等待一段时间或需要等待一个事件发生再运行时，该任务就会把 CPU 的使用权让给别的任务而使任务进入等待状态。

中断服务状态，一个正在运行的任务一旦响应中断申请就会中止运行而去执行中断服务程序，这时任务的状态叫做中断服务状态。

20.2 UCOSII任务（续）

➢ **任务状态转移：**



参考文献：任哲，《嵌入式实时操作系统 UCOSII 原理及应用》（第6版），北京航空航天大学出版社，2024

20.3 与任务相关的函数

1）建立任务函数

OSTaskCreate(void(*task)(void*pd),void*pdata,OS_STK*ptos,INTU prio)

task：指向任务代码的指针；

pdata：任务开始执行时，传递给任务的参数的指针；

ptos：分配给任务的堆栈的栈顶指针；每个任务都有自己的堆栈，堆栈必须申明为 OS_STK 类型，并且由连续的内存空间组成。可以静态分配堆栈空间，也可以动态分配堆栈空间。

prio ：分配给任务的优先级。

OSTaskCreateExt 函数也可以用来创建任务，是 OSTaskCreate 的扩展版本，提供一些附加功能。

20.3 与任务相关的函数（续）

2） 任务删除函数

把任务置于睡眠状态，并不是把任务代码给删除了。

INT8U OSTaskDel(INT8U prio);

prio： 要删除的任务的优先级。

特别注意：任务不能随便删除，必须在确保被删除任务的资源被释放的前提下才能删除！！！

20.3 与任务相关的函数（续）

3） 请求任务删除函数

该函数请求任务释放自身占用的资源，为任务删除做准备。

INT8U OSTaskDelReq(INT8U prio);

prio： 要删除的任务的优先级。

4） 改变任务的优先级函数

INT8U OSTaskChangePrio(INT8U oldprio,INT8U newprio);

oldprio： 任务的旧优先级。

newprio： 任务的新优先级。

20.3 与任务相关的函数（续）

5） 任务挂起函数

任务挂起和任务删除有点类似，但是又有区别，任务挂起只是将被挂起任务的就绪标志删除，并做任务挂起记录，并没有将任务控制块任务控制块链表里面删除，也不需要释放其资源，而任务删除则必须先释放被删除任务的资源，并将被删除任务的任务控制块也给删了。被挂起的任务，在恢复（解挂）后可以继续运行。

INT8U OSTaskSuspend(INT8U prio);

prio： 任务的优先级。

20.3 与任务相关的函数（续）

6） 任务恢复函数

该函数将被挂起的任务恢复，让调度器能够重新调度该任务。

INT8U OSTaskResume(INT8U prio);

prio： 任务的优先级。

7） 任务信息查询

获取对应任务的 OS_TCB 中内容的拷贝。

INT8U OSTaskQuery(INT8U prio,OS_TCB *pdata);

prio： 任务的优先级。

pdata ： 目的数据块指针

20.4 STM32上使用UCOSII

1） UCOSII移植（略）

2） 编写任务函数

设置函数堆栈大小，根据函数的需求来设置。注意堆栈字节对齐的问题，如果任务运行出现莫名其妙的错误（比如用到 sprintf 出错），请考虑是不是字节对齐的问题。

设置任务优先级，根据任务的重要性和实时性设置

20.4 STM32上使用UCOSII（续）

3）初始化 UCOSII ，并在 UCOSII 中创建任务

调用 OSInit，初始化 UCOSII；通过调用 OSTaskCreate 函数创建任务

4） 启动 UCOSII

调用 OSStart，启动 UCOSII。

20.5 UCOSII示例1-任务调度

1. 功能要求： 基于UCOSII操作系统，实现LED灯闪烁控制：DS0每秒钟亮 80ms； DS1 亮 300ms，灭 300ms，依次循环。

2. 实现思路： 在 UCOSII 里面创建 3 个任务：开始任务、LED0任务和 LED1任务

开始任务：创建其他两个任务，之后挂起；

LED0 任务：用于控制 DS0的亮灭；

LED1 任务：用于控制 DS1 的亮灭

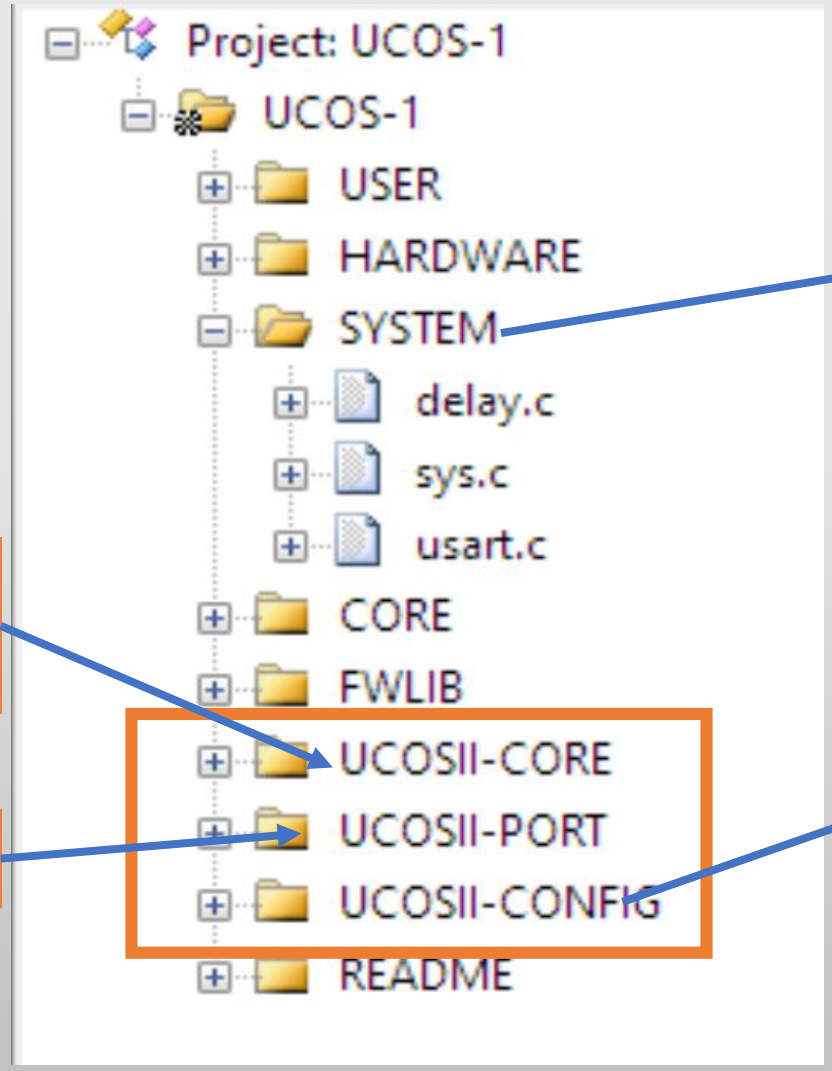
20.5 UCOSII示例1-任务调度（续）

3. 硬件电路

20.5 UCOSII示例1-任务调度（续）

4. 软件设计

➢ 添加UCOSII相关文件



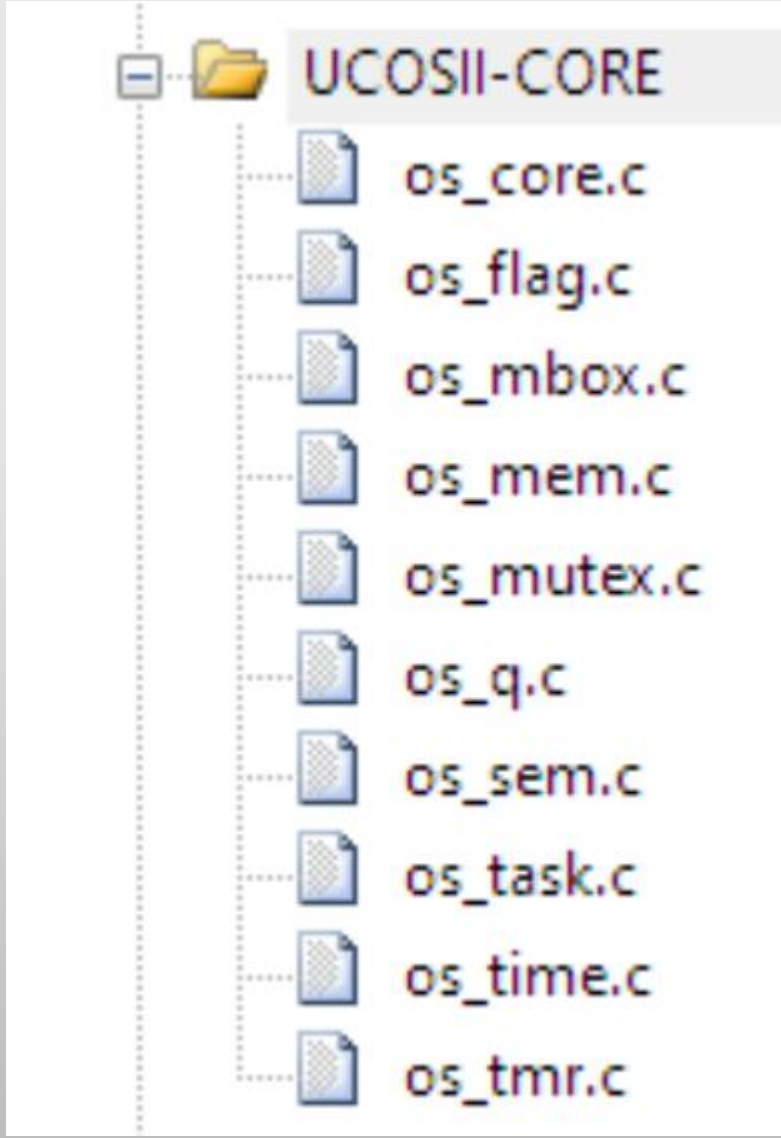
支持OS

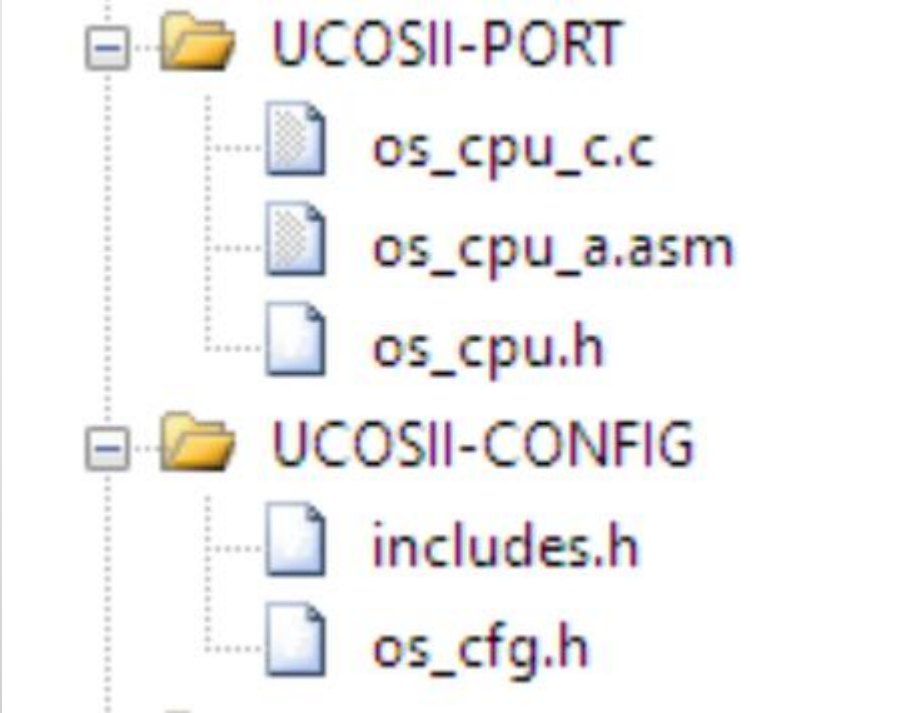
UCOSII 的核心源码，不需要做任何变动

需要修改

用户根据需要进行裁剪

UCOSII相关文件


UCOSII-CORE
os_core.c
os_flag.c
os_mbox.c
os_mem.c
os_mutex.c
os_q.c
os_sem.c
os_task.c
os_time.c
os_tmr.c


UCOSII-PORT
os_cpu_c.c
os_cpu_a.asm
os_cpu.h
UCOSII-CONFIG
includes.h
os_cfg.h

Includes.h文件

```
9
10 #ifndef __INCLUDES_H__
11 #define __INCLUDES_H__
12 #include <stdio.h>
13 #include <string.h>
14 #include <ctype.h>
15 #include <stdlib.h>
16 #include <stdarg.h>
17
18 #include "ucos_ii.h"
19 #include "os_cpu.h"
20 #include "os_cfg.h"
21
22 #include <stm32f4xx.h>
23
24 #endif
25
```

os_cfg.h文件

最大任务数量，至少>=2

```
#define OS_MAX_QS              5u   /* Max. number of queue control blocks in your application
#define OS_MAX_TASKS          10u   /* Max. number of tasks in your application, MUST be >= 2

#define OS_SCHED_LOCK_EN       1u   /* Include code for OSSchedLock() and OSSchedUnlock()

#define OS_TICK_STEP_EN        1u   /* Enable tick stepping feature for uC/OS-View
#define OS_TICKS_PER_SEC     200u   /* Set the number of ticks in one second
```

每秒ticks数量，=200

- UCOSII 的时钟节拍由 SYSTICK 的中断服务函数提供

void delay_init(u8 SYSCLK)：确定节拍时间

```
#define delay_osticksperseс OS_TICKS_PER_SEC  //OS时钟节拍,即每秒调度次数
```

设SYSTICK频率=168MHZ/8=21MHz

```
//初始化延迟函数
//当使用OS的时候,此函数会初始化OS的时钟节拍
//SYSTICK的时钟固定为AHB时钟的1/8
//SYSCLK:系统时钟频率
void delay_init(u8 SYSCLK)
{
#if SYSTEM_SUPPORT_OS                    //如果需要支持OS.
  u32 reload;
#endif
  SysTick_CLKSourceConfig(SysTick_CLKSource_HCLK_Div8);
  fac_us=SYSCLK/8;                  //不论是否使用OS,fac_us都需要使用
#if SYSTEM_SUPPORT_OS                    //如果需要支持OS.
  reload=SYSCLK/8;                  //每秒钟的计数次数 单位为M
  reload*=1000000/delay_osticksperseс;  //根据delay_osticksperseс设定溢出时间
                    //reload为24位寄存器,最大值:16777216,在168M下,约合0.7989s左右
  fac_ms=1000/delay_osticksperseс;     //代表OS可以延时的最少单位 =?
  SysTick->CTRL|=SysTick_CTRL_TICKINT_Msk;    //开启SYSTICK中断
  SysTick->LOAD=reload;                //每1/delay_osticksperseс秒中断一次
  SysTick->CTRL|=SysTick_CTRL_ENABLE_Msk;   //开启SYSTICK
#else
  fac_ms=(u16)fac_us*1000;             //非OS下,代表每个ms需要的systick时钟数
#endif
}
```

=200

- UCOSII 的时钟节拍由 SYSTICK 的中断服务函数提供

```
//systick中断服务函数,使用os时用到
void SysTick_Handler(void)
{
  if(delay_osrunning==1)   //os开始跑了,才执行正常的调度处理
  {
    OSIntEnter();            //进入中断
    OSTimeTick();            //调用ucos的时钟服务程序
    OSIntExit();             //触发任务切换软中断
  }
}
```

OSIntEnter：是进入中断服务函数，用来记录中断嵌套层数（OSIntNesting增加 1）；

OSTimeTick：系统时钟节拍服务函数，在每个时钟节拍了解每个任务的延时状态，使已经到达延时时限的非挂起任务进入就绪状态；

OSIntExit：退出中断服务函数，该函数可能触发一次任务切换（当 OSIntNesting==0&&调度器未上锁&&就绪表最高优先级任务！=被中断的任务优先级时），否则继续返回原来的任务执行代码（如果 OSIntNesting 不为 0，则减 1）。

注意！！！：任何中断服务函数，都应该加上 OSIntEnter 和OSIntExit 函数，这是因为UCOSII 是一个可剥夺型的内核，中断服务子程序运行之后，系统会根据情况进行一次任务调度去运行优先级别最高的就绪任务，而并不一定接着运行被中断的任务！

- 其他：可提供us级精确延时（delay.c中）

main.c代码

```
//////////////////////////UCOSII任务设置////////////////////////////////
//START 任务
//设置任务优先级
#define START_TASK_PRIO          10 //开始任务的优先级设置为最低
//设置任务堆栈大小
#define START_STK_SIZE          64
//任务堆栈
OS_STK START_TASK_STK[START_STK_SIZE];
//任务函数
void start_task(void *pdata);

//LED0任务
//设置任务优先级
#define LED0_TASK_PRIO           7
//设置任务堆栈大小
#define LED0_STK_SIZE           64
//任务堆栈
OS_STK LED0_TASK_STK[LED0_STK_SIZE];
//任务函数
void led0_task(void *pdata);


//LED1任务
//设置任务优先级
#define LED1_TASK_PRIO           6
//设置任务堆栈大小
#define LED1_STK_SIZE           64
//任务堆栈
OS_STK LED1_TASK_STK[LED1_STK_SIZE];
//任务函数
void led1_task(void *pdata);
```

主函数-任务设置

main.c代码（续）

主函数：硬件初始化，OS初始化，创建start_task,启动OS

```
int main(void)
{

  delay_init(168);        //初始化延时函数
  LED_Init();             //初始化LED端口
  OSInit();
  OSTaskCreate(start_task,(void *)0,(OS_STK *)&START_TASK_STK[START_STK_SIZE-1],START_TASK_PRIO );//创建起始任务
  OSStart();
}
```

main.c代码（续）

```
//开始任务
void start_task(void *pdata)
{
    OS_CPU_SR cpu_sr=0;
  pdata = pdata;
    OS_ENTER_CRITICAL();        //进入临界区(无法被中断打断)
  OSTaskCreate(led0_task,(void *)0,(OS_STK*)&LED0_TASK_STK[LED0_STK_SIZE-1],LED0_TASK_PRIO);
  OSTaskCreate(led1_task,(void *)0,(OS_STK*)&LED1_TASK_STK[LED1_STK_SIZE-1],LED1_TASK_PRIO);
  OSTaskSuspend(START_TASK_PRIO); //挂起起始任务.
  OS_EXIT_CRITICAL();         //退出临界区(可以被中断打断)
}
```

start_task：初始化统计任务；进入临界区；创建其他任务；退出临界区

有一些代码段必须不受任何干扰地连续运行，这样的代码段叫做临界段或**临界区**

单独创建 start_task，是为了提供一个单一任务，实现应用程序开始运行之前的准备工作（比如：外设初始化、创建信号量、创建邮箱、创建消息队列、创建信号量集、创建任务、初始化统计任务等等）。

main.c代码（续）

```
void led0_task(void *pdata)
{
  while(1)
  {
    LED0=0;    delay_ms(80);
    LED0=1;    delay_ms(920);
  };
}
```

LED0任务

```
void led1_task(void *pdata)
{
  while(1)
  {
    LED1=0;    delay_ms(300);
    LED1=1;    delay_ms(300);
  };
}
```

LED1任务

20.6 UCOSII示例2-任务删除，挂起和恢复

1. 功能要求：3 个任务:

led_task : LED0 和 LED1 每隔 500ms 翻转一次。

beep_task: 在没有收到删除请求的时候是隔一段时间蜂鸣器鸣叫一次

key_task : 按键扫描。

KEY_RIGHT 按键按下时挂起任务 led_task，这时 LED0 和 LED1 停止闪烁。

当 KEY_LEFT 按键按下时，如果 led_task 被挂起则恢复之，如果没有挂起则没有影响。

当 KEY_UP 按键按下时删除任务 beep_task。

当 KEY1 按键按下时，重新创建任务 beep_task。

main.c文件代码

主函数-任务设置

任务名称	任务优先级	任务堆栈大小
start_task	10	64
led_task	7	64
beep_task	5	64
key_task	3	64

main.c文件代码（续）

主函数：硬件初始化，OS初始化，创建start_task,启动OS

```
int main(void)
{

    delay_init(168);            //初始化延时函数
    uart_init(115200);
    LED_Init();                 //初始化与 LED 连接的硬件接口
    BEEP_Init();                //蜂鸣器初始化
    KEY_Init();                 //按键初始化
    OSInit();                   //初始化 UCOSII
    OSTaskCreate(start_task,(void *)0,(OS_STK *)&START_TASK_STK
[START_STK_SIZE-1],START_TASK_PRIO );//创建起始任务
    OSStart();
}
```

main.c文件代码（续）

start_task：初始化统计任务；进入临界区；创建其他任务；退出临界区

```
void start_task(void *pdata)
{
        OS_CPU_SR cpu_sr=0;
        pdata = pdata;
        OSStatInit();                          //初始化统计任务.这里会延时 1 秒钟左右
        OS_ENTER_CRITICAL();                   //进入临界区(无法被中断打断)
        OSTaskCreate(led_task,(void *)0,(OS_STK*)&LED_TASK_STK
                        [LED_STK_SIZE-1],LED_TASK_PRIO);
        OSTaskCreate(beep_task,(void *)0,(OS_STK*)&BEEP_TASK_STK
                        [BEEP_STK_SIZE-1],BEEP_TASK_PRIO);
        OSTaskCreate(key_task,(void *)0,(OS_STK*)&KEY_TASK_STK
                        [KEY_STK_SIZE-1],KEY_TASK_PRIO);
        OSTaskSuspend(START_TASK_PRIO);        //挂起起始任务.
        OS_EXIT_CRITICAL();                    //退出临界区(可以被中断打断)
}
```

main.c文件代码（续）

led_task

```
void led_task(void *pdata)
{
    while(1)
    {
        LED0=!LED0;
        LED1=!LED1;delay_ms(500);
    }
}
```

main.c代码代码（续）

beep_task

```
void beep_task(void *pdata)
{
        while(1)
        {
  if(OSTaskDelReq(OS_PRIO_SELF)==OS_ERR_TASK_DEL_REQ) //判断是否有删除请求
                {
                OSTaskDel(OS_PRIO_SELF);                //删除任务本身
                }
                BEEP=1; delay_ms(60);
                BEEP=0; delay_ms(940);
        }
}
```

main.c代码（续）

key_task

```
void key_task(void *pdata){
    u8 key;
    while(1){
        key=KEY_Scan(0);
        if(key==KEY_RIGHT) {   //挂起 LED 任务，LED 停止闪烁
        OSTaskSuspend(LED_TASK_PRIO);}
        else if (key==KEY_LEFT){          //恢复 LED 任务，LED 恢复
        闪烁
        OSTaskResume(LED_TASK_PRIO);}
        else if (key==KEY_UP){            //发送删除 BEEP 任务请求
        OSTaskDelReq(BEEP_TASK_PRIO); }
        else if(key==KEY_DOWN){                   //重新创建任务
        beep
        OSTaskCreate(beep_task,(void
        *)0,(OS_STK*)&BEEP_TASK_STK
        [BEEP_STK_SIZE-1],BEEP_TASK_PRIO);}
        delay_ms(10);}
}
```

20.7 UCOSII示例3-信号量（Semaphore）和邮箱（Box）

1. 简介

操作系统多任务协调管理能力

例如：任务A和任务B共享一台打印机

- 多任务资源共享

  需要互相无冲突地访问同一个共享资源

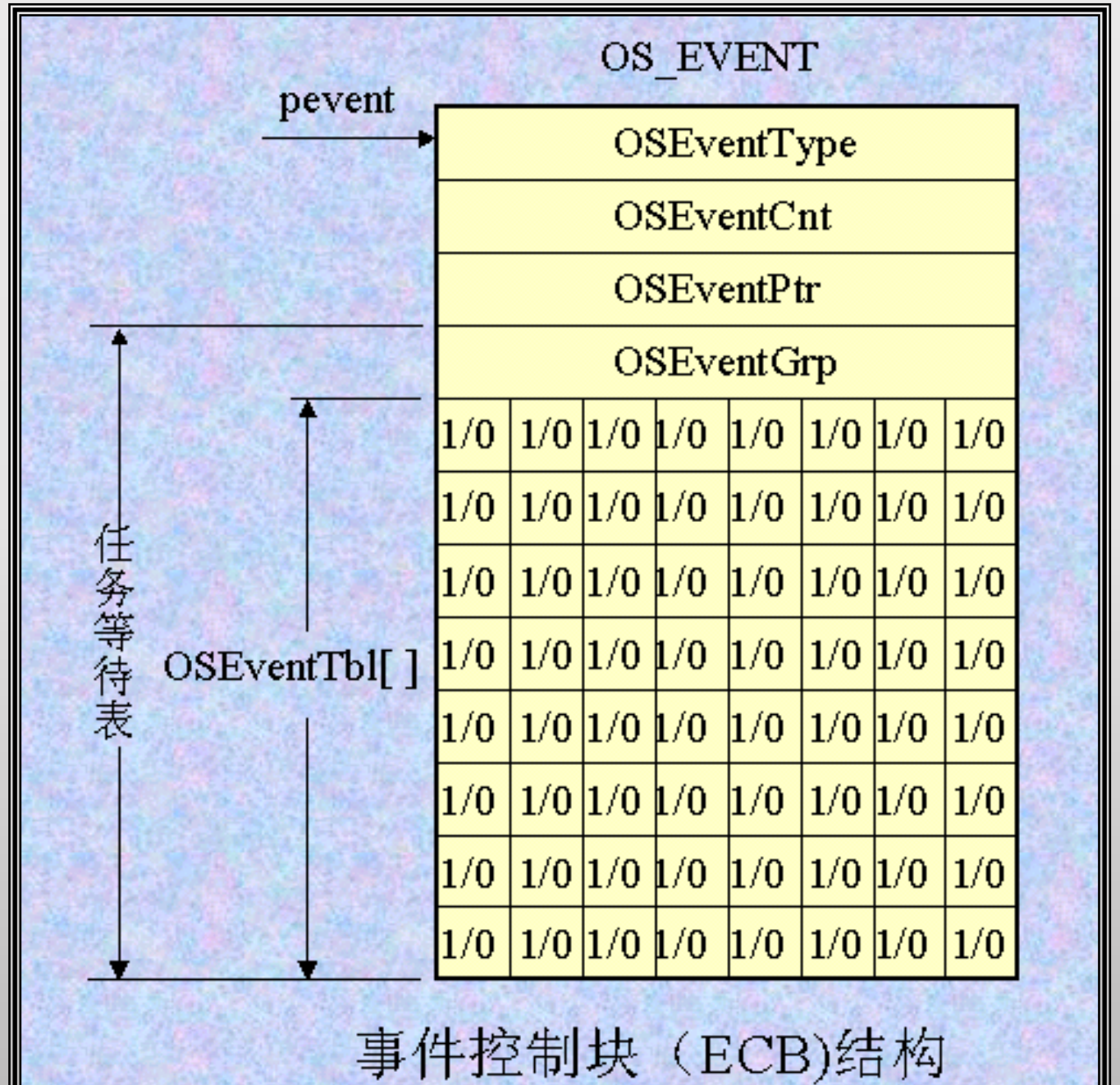
- 多任务互相支持、依赖、限制、制约
- 操作系统必须具有对任务的运行进行协调的能力，从而使任务之间可以无冲突、流畅地同步运行，而不致导致灾难性的后果。

- 任务间的同步依赖于任务间的通信。
- 在 UCOSII 中，任务间通信使用信号量、邮箱（消息邮箱）和消息队列 （都被称作事件Event）来实现

读事件操作叫做请求事件：如果有信息则读取，否则等待

➢ 事件控制块(ECB)结构体：

```
typedef struct{
INT8U OSEventType; //事件的类型
INT16U OSEventCnt;  //信号量计数器
void *OSEventPtr; //消息或消息队列的指针
INT8U OSEventGrp;  //等待事件的任务组
INT8U OSEventTbl[OS_EVENT_TBL_SIZE];//任务等待表
#if OS_EVENT_NAME_EN > 0u
INT8U *OSEventName; //事件名
#endif
} OS_EVENT;
```



事件类型：

信号量　OS_EVENT_TYPE_SEM、
互斥信号量 OS_EVENT_TYPE_MUTEX、
消息邮箱　OS_EVENT_TYPE_MBOX、
消息队列　OS_EVENT_TYPE_Q、
信号量集　OS_EVNET_TYPE_FLAG、

信号量、消息和消息队列区别

1、用信号量进行行为同步时，只能提供同步的**时刻信息**，不能提供内容信息。若被控制方要求得到控制方的内容信息时，可以使用消息邮箱或消息队列。

2、消息邮箱里只能存放**一条消息**，所以使用消息邮箱进行任务的同步时，需要满足一个条件：消息的产生速度总要慢于消息的消费速度，即被控制任务总是在等待消息，否则会导致消息丢失。

3、若遇到出现消息的产生速度可能快于消息的消费速度的情况时，则可以使用比消息邮箱更为强大的消息队列，由于消息队列可以存放多条消息，所以消息队列能够有效解决消息的**临时堆积问题**。但消息队列的使用仍然需满足一个条件：消息的平均生产速率比消息的**平均消费速率**低，否则再长的消息队列也会溢出。

20.7 UCOSII示例3-信号量和邮箱（续）

2. 信号量

- **信号量目的：**给共享资源设立一个标志，该标志表示该共享资源的占用情况。

  当一个任务在访问共享资源之前，就可以先对这个标志进行查询，从而在了解资源被占用的情况之后，再来决定自己的行为。

20.7 UCOSII示例3-信号量和邮箱（续）

- **信号量分类：** 二值型信号量，N 值信号量。

二值型信号量（互斥型信号量）： 任何时候，只能有一个任务占用。

N 值信号量（计数型信号量）： 同时有多个任务（N 个）使用，也就是普通的信号量。

本例题：普通信号量

20.7 UCOSII示例3-信号量和邮箱（续）

3. UCOS中与信号量相关的重要函数

1） 创建信号量

在使用信号量之前，必须用函数 OSSemCreate 来创建一个信号量：

OS_EVENT *OSSemCreate (INT16U cnt);

返回值：已创建的信号量的指针

参数cnt：信号量计数器（OSEventCnt）的初始值

2） 请求信号量：

void OSSemPend (OS_EVENT *pevent, INT16U timeout, INT8U *err);

pevent ： 被请求信号量的指针，

timeout ： 等待时限　为防止任务因得不到信号量而处于长期的等待状态，函数 OSSemPend 允许用参数timeout 设置一个等待时间的限制，当任务等待的时间超过 timeout 时可以结束等待状态而进入就绪状态。如果参数 timeout 被设置为 0，则表明任务的等待时间为无限长。

err： 为错误信息。

3） 发送信号量（释放信号量）

INT8U OSSemPost(OS_EVENT *pevent);

pevent：信号量指针，

返回值：若调用成功，返回 OS_NO_ERR，否则会根据具体错误返回 OS_ERR_EVENT_TYPE、OS_SEM_OVF。

4） 删除信号量

应用程序如果不需要某个信号量，可以删除该信号量：

```
OS_EVENT *OSSemDel (OS_EVENT *pevent,INT8U opt, INT8U *err);
```

pevent：要删除的信号量指针，

opt ： 删除条件选项，

err ： 错误信息。

20.7 UCOSII示例3-信号量和邮箱（续）

4. 邮箱

在多任务操作系统中，常常需要在任务与任务之间通过传递一个数据（“消息”）的方式来进行通信。为此，可以在内存中创建一个存储空间作为该数据的缓冲区（消息缓冲区），这样在任务间传递数据（消息）的最简单办法就是传递消息缓冲区的指针。用来传递消息缓冲区指针的数据结构叫做邮箱（消息邮箱）。

在 UCOSII 中，通过事件控制块的 OSEventPrt 来传递消息缓冲区指针，同时使事件控制块的成员 OSEventType 为常数 OS_EVENT_TYPE_MBOX，则该事件控制块就叫做消息邮箱

20.7 UCOSII示例3-信号量和邮箱（续）

5. UCOS中邮箱相关的重要函数

1） **创建邮箱**：创建一个邮箱

OS_EVENT *OSMboxCreate (void *msg);

msg ：消息的指针

返回值：消息邮箱的指针。

调用函数 OSMboxCreate 需先定义 msg 的初始值。一般，这个初始值为NULL；但也可以事先定义一个邮箱，然后把这个邮箱的指针作为参数传递到函数OSMboxCreate 中，使之一开始就指向一个邮箱。

2）**向邮箱发送消息**：将消息发送到指定的消息邮箱中

INT8U OSMboxPost (OS_EVENT *pevent,void *msg);

pevent ： 消息邮箱的指针

Msg：消息指针。

3） 请求邮箱函数

查看邮箱指针 OSEventPtr 是否为 NULL，如果不是 NULL 就把邮箱中的消息指针返回给调用函数的任务，同时用 OS_NO_ERR 通过函数的参数 err 通知任务获取消息成功；如果邮箱指针OSEventPtr 是 NULL，则使任务进入等待状态，并引发一次任务调度。

void *OSMboxPend (OS_EVENT *pevent, INT16U timeout, INT8U *err);

Pevent：请求邮箱指针

timeout ：等待时限

err：错误信息。

4） 查询邮箱状态函数

INT8U OSMboxQuery(OS_EVENT *pevent,OS_MBOX_DATA *pdata);

pevent：消息邮箱指针

pdata ：存放邮箱信息的结构（局部变量）。

```
typedef struct os_mbox_data {

    void   *OSMsg; /* 指向邮箱中消息的指针  */
    OS_PRIO OSEventTbl[OS_EVENT_TBL_SIZE]; /* 消息的等待任务列表  */
    OS_PRIO OSEventGrp; /* 消息的等待任务所在的组  */
} OS_MBOX_DATA;
```

5） 删除邮箱函数

OS_EVENT *OSMboxDel(OS_EVENT *pevent,INT8U opt,INT8U *err);

pevent：消息邮箱指针

opt ： 删除选项

err ： 错误信息。

20.7 UCOSII示例3-信号量和邮箱（续）

6. 邮箱示例

功能要求：

创建 6 个任务：开始任务、LED 任务、触摸屏任务、蜂鸣器任务、按键扫描任务和主任务

①开始任务：用于创建信号量、创建邮箱、初始化统计任务以及其他任务的创建，之后挂起；

② LED 任务： DS0 控制，提示程序运行状况；

③蜂鸣器任务：测试信号量，是请求信号量函数，每得到一个信号量，蜂鸣器就叫一次；

④按键扫描任务：按键扫描，优先级最高，将得到的键值通过消息邮箱发送出去；

⑤主任务：通过查询消息邮箱获得键值，并根据键值执行 LED 控制、信号量发送（蜂鸣器控制）、触摸区域清屏和触摸屏校准等控制。

main.c文件代码

主函数-任务设置

任务名称	任务优先级	任务堆栈大小
start_task	10	64
led_task	6	64
beep_task	5	64
main_task	4	128
key_task	3	64

```
//////////////////////////////////////////////////////////////////////////////
OS_EVENT * msg_key;          //按键邮箱事件块指针
OS_EVENT * sem_beep;         //蜂鸣器信号量指针
```

main.c文件代码（续）

主函数：硬件初始化，OS初始化，创建start_task,启动OS

```
int main(void)
{
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);//设置系统中断优先级分组 2
    delay_init(168); //初始化延时函数
    uart_init(115200); //初始化串口波特率为 115200
    LED_Init();  //初始化 LED
    BEEP_Init(); //蜂鸣器初始化
    KEY_Init(); //按键初始化
    LCD_Init();  //LCD初始化
    OSInit(); //初始化 UCOSII
    OSTaskCreate(start_task,(void *)0,(OS_STK *)&START_TASK_STK[START_STK_SIZE-1],
                 START_TASK_PRIO );//创建起始任务
    OSStart();  //启动 UCOSII
}
```

main.c文件代码（续）

start_task：初始化变量；初始化统计任务；进入临界区；创建其他任务；退出临界区

```
//开始任务
void start_task(void *pdata)
{
OS_CPU_SR cpu_sr=0;
pdata = pdata;
msg_key=OSMboxCreate((void*)0);//创建按键邮箱msg_key
sem_beep=OSSemCreate(0); //创建蜂鸣器信号量msg_beep
OSStatInit(); //初始化统计任务.这里会延时 1 秒钟左右
OS_ENTER_CRITICAL(); //进入临界区(无法被中断打断)
OSTaskCreate(led_task,(void *)0,(OS_STK*)&LED_TASK_STK[LED_STK_SIZE-1],LED_TASK_PRIO);
OSTaskCreate(beep_task,(void *)0,(OS_STK*)&BEEP_TASK_STK[BEEP_STK_SIZE-1],BEEP_TASK_PRIO);
OSTaskCreate(main_task,(void *)0,(OS_STK*)&MAIN_TASK_STK[MAIN_STK_SIZE-1],MAIN_TASK_PRIO);
OSTaskCreate(key_task,(void *)0,(OS_STK*)&KEY_TASK_STK[KEY_STK_SIZE-1],KEY_TASK_PRIO);
OSTaskSuspend(START_TASK_PRIO);  //挂起起始任务.
OS_EXIT_CRITICAL();  //退出临界区(可以被中断打断)
}
```

main.c文件代码（续）

led_task

```
//LED 任务
void led_task(void *pdata)
{
    u8 t;
    while(1)
    {
        t++;delay_ms(10);
        if(t==8)LED0=1;    //LED0 灭
        if(t==100)         //LED0 亮
        {
            t=0;LED0=0;
        }
    }
}
```

main.c文件代码（续）

```
//蜂鸣器任务
void beep_task(void *pdata)
{
        u8 err;
        while(1)
        {
                OSSemPend(sem_beep,0,&err);//请求蜂鸣器信号量sem_beep；
                BEEP=1;delay_ms(60);          //蜂鸣器鸣叫
                BEEP=0;delay_ms(940);         //蜂鸣器停止鸣叫
        }
}
```

main.c文件代码（续）

key_task:

```
//按键扫描任务
void key_task(void *pdata)
{
        u8 key;
        while(1)
        {
                key=KEY_Scan(0);                //按键扫描;
                if(key)OSMboxPost(msg_key,(void*)key);//如果有键按下，发送键号给按键邮箱msg_key
                delay_ms(10);
        }
}
```

main.c文件代码（续）

main_task

```
//主任务
void main_task(void *pdata){
    u32 key=0;
    u8 err,semmask=0,tcnt=0;
    while(1){
        key=(u32)OSMboxPend(msg_key,10,&err);  //请求按键邮箱msg_key
        switch(key){
            case 1:  LED1=!LED1;  break; //控制 DS1
            case 2:  semmask=1;  OSSemPost(sem_beep); break; }//发送蜂鸣器信号量sem_beep
        if(semmask||sem_beep->OSEventCnt) {            //LCD显示 sem_beep次数
            LCD_show(sem_beep>OSEventCnt);
            if(sem_beep->OSEventCnt==0)semmask=0; }  //停止更新
        }
    delay_ms(10);
}
```

20.8 UCOSII示例4-消息队列、信号量集和软件定时器

1. 消息队列

使用消息队列可在任务间传递多条消息

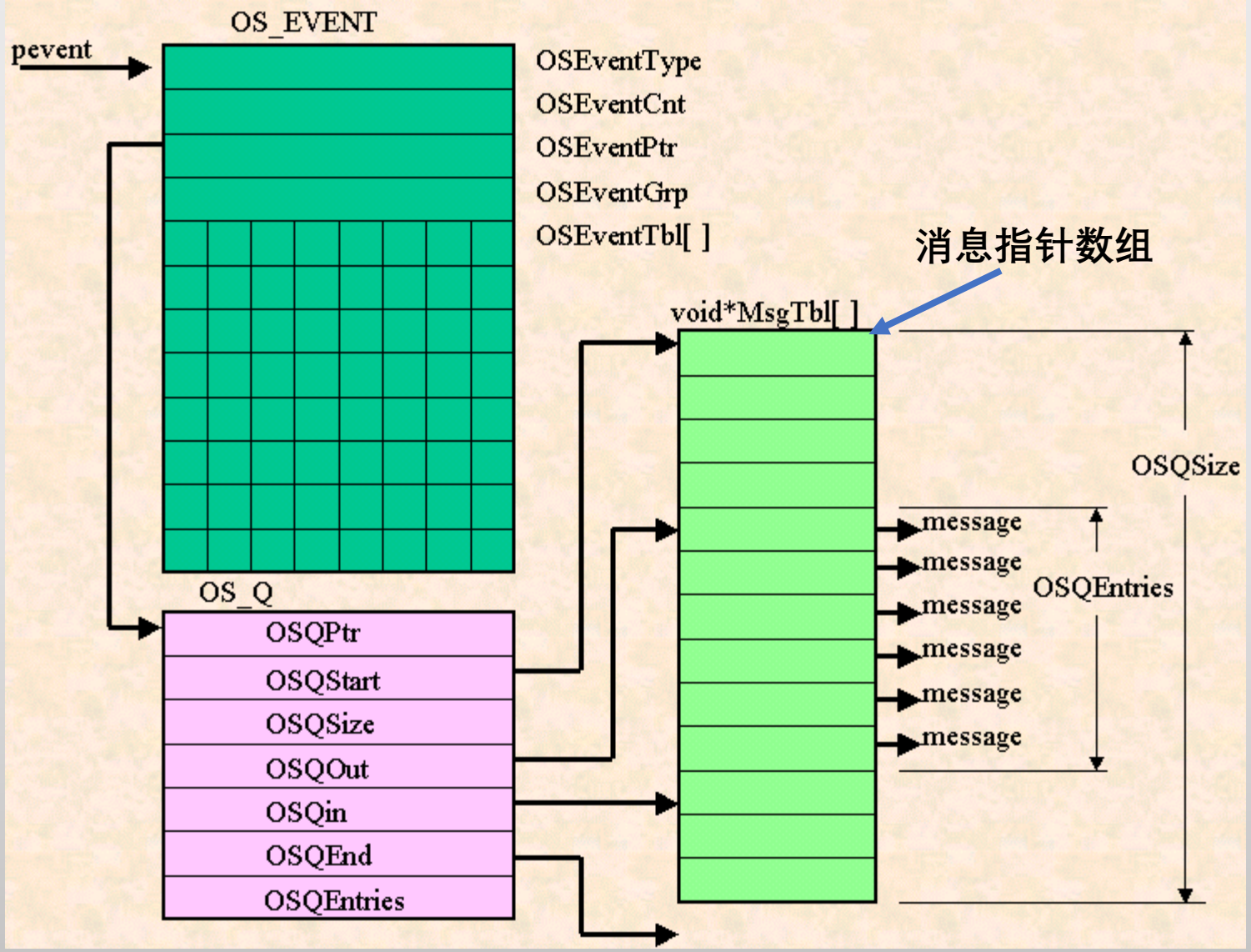
消息队列：使用消息队列可以在任务之间传递多条消息。

消息队列由三个部分组成：事件控制块、消息队列和消息。

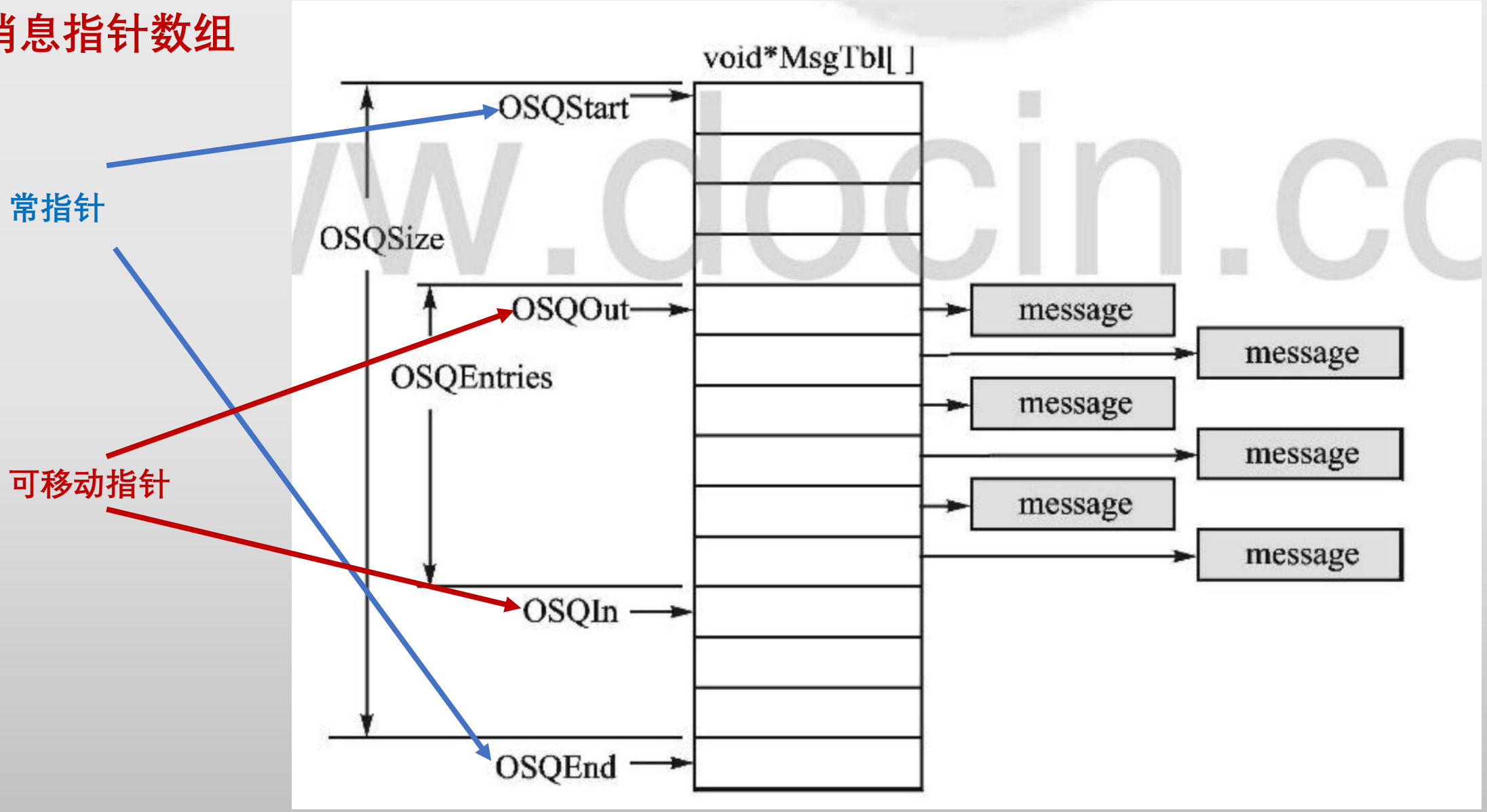
当把事件控制块成员 OSEventType = OS_EVENT_TYPE_Q 时，该事件控制块描述的就是一个消息队列。

消息队列的数据结构

消息队列是共用一个任务等待列表的**消息邮箱数组**，事件控制块成员 OSEventPtr 指向了一个叫做队列控制块（OS_Q）的结构，该结构管理了一个消息邮箱数组 MsgTbl[]，该数组中的元素都是一些指向消息的指针。

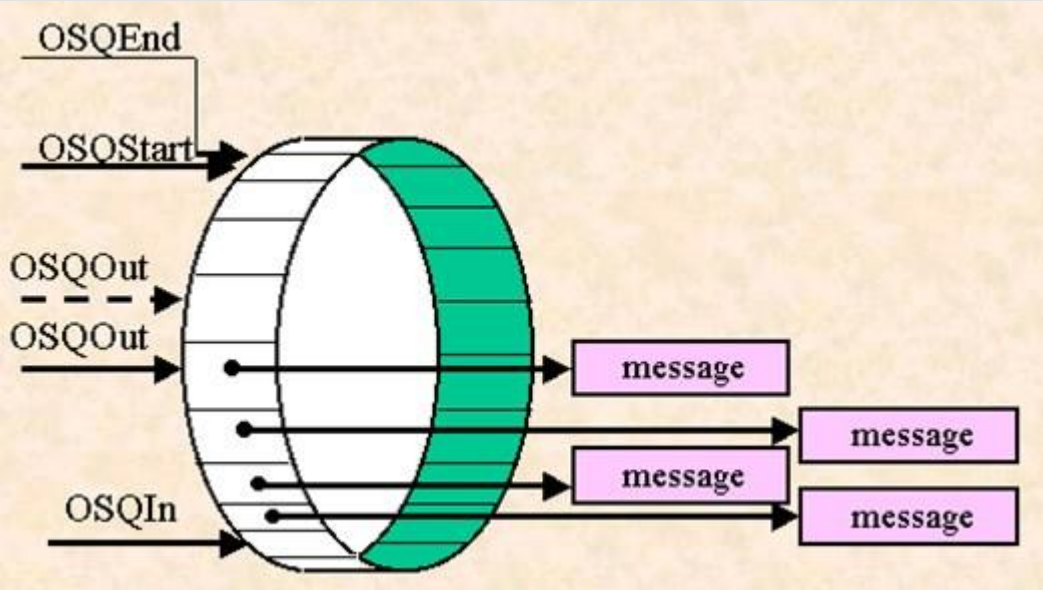


消息指针数组


常指针
可移动指针
void*MsgTbl[ ]
OSQStart
OSQSize
OSQOut
OSQEntries
OSQIn
OSQEnd
message
message
message
message
message

消息指针数组---环形队列

当可移动的指针 OSQIn 或 OSQOut 移动到数组末尾，也就是与 OSQEnd 相等时，可移动的指针将会被调整到数组的起始位置 OSQStart。也就是说，从效果上来看，指针 OSQEnd与 OSQStart 等值。于是，这个由消息指针构成的数组就头尾衔接起来形成了一个如图所示的循环的队列。



队列控制块OS_Q：

```
typedef struct os_q
{
struct os_q *OSQPtr; //指向下一个空的os_q
void **OSQStart; //指向消息指针数组的起始地址
void **OSQEnd;      //指向消息指针数组结束单元的下一个单元。它使得数组构成了一个循环
的缓冲区
void **OSQIn; //指向插入一条消息的位置。当它移动到与 OSQEnd 相等时，被调整到指向数组的起始单元
void **OSQOut;  //指向被取出消息的位置。当它移动到与 OSQEnd 相等时，被调整到指向数组的起始单元
INT16U OSQSize; //数组的长度
INT16U OSQEntries;  //已存放消息指针的元素数目
} OS_Q;
```

在UCOSII初始化时，系统将按文件os_cfg.h中的配置常数OS_MAX_QS定义OS_MAX_QS个队列控制块，并用队列控制块中的指针 OSQPtr 将所有队列控制块链接为链表。由于这时还没有使用它们，故这个链表叫做空队列控制块链表。

消息队列相关的部分函数

1） 创建消息队列

创建一个消息队列首先需要定义一指针数组，然后把各个消息数据缓冲区的首地址存入这个数组中，然后再调用函数 OSQCreate 来创建消息队列。

OS_EVENT *OSQCreate(void**start,INT16U size);

其中，start 为存放消息缓冲区指针数组的地址，size 为该数组大小。该函数的返回值为消息队列指针。

消息队列相关的部分函数

2） 请求消息队列

请求消息队列的目的是为了从消息队列中获取消息。

void*OSQPend(OS_EVENT*pevent,INT16U timeout,INT8U *err);

其中，pevent 为所请求的消息队列的指针，timeout 为任务等待时限，err 为错误信息。

消息队列相关的部分函数

3） 向消息队列发送消息

任务可以通过调用函数 OSQPost 或 OSQPostFront 两个函数来向消息队列发送消息。

函数 OSQPost 以 FIFO（先进先出）的方式组织消息队列，函数 OSQPostFront 以 LIFO（后进先出）的方式组织消息队列。这两个函数的原型分别为：

```
INT8U OSQPost(OS_EVENT*pevent,void *msg);
INT8U OSQPostFront (OS_EVENT*pevent,void*msg);
```

其中，pevent 为消息队列的指针，msg 为待发消息的指针。

20.8 UCOSII示例4-消息队列、信号量集和软件定时器（续）

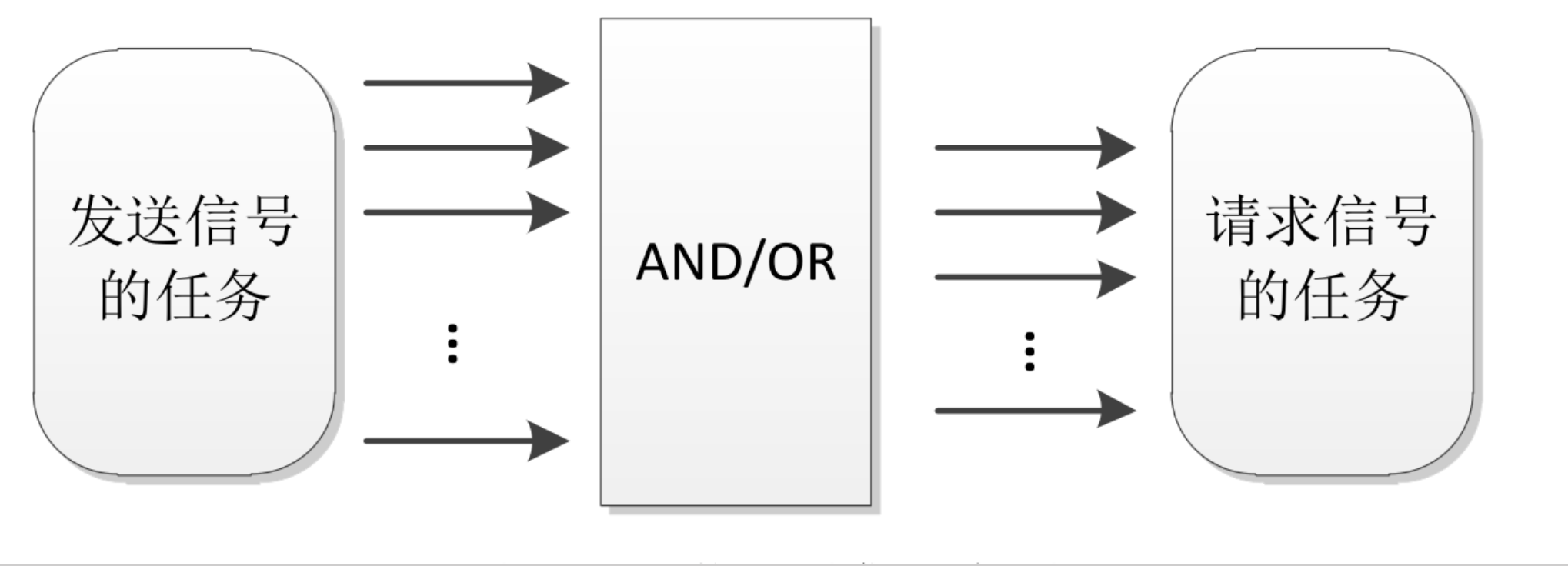
2. 信号量集

在实际应用中，任务常常需要与多个事件同步，即要根据多个信号量组合作用的结果来决定任务的运行方式。

UCOSII 为了实现多个信号量组合的功能定义了一种特殊的数据结构——信号量集。

信号量集所能管理的信号量都是一些二值信号，所有信号量集实质上是一种可以对多个输入的逻辑信号进行基本逻辑运算的组合逻辑

信号量集示意图



UCOSII 使用了一个叫做标志组的结构 OS_FLAG_GRP 来描述信号量集。

```
typedef struct
{
INT8U  OSFlagType; //识别是否为信号量集的标志
void *OSFlagWaitList; //指向等待任务链表的指针
OS_FLAGS OSFlagFlags; //所有信号列表， 8 位/16 位（默认）/32 位
}OS_FLAG_GRP;
```

成员 OSFlagWaitList 是一个指针，当一个信号量集被创建后，这个指针指向了这个信号量集的等待任务链表。与其他前面介绍过的事件不同，信号量集用一个双向链表来组织等待任务，每一个等待任务都是该链表中的一个节点（Node）。

等待任务链表节点 OS_FLAG_NODE 的结构

```
typedef struct
{
    void      *OSFlagNodeNext;        //指向下一个节点的指针
    void      *OSFlagNodePrev;        //指向前一个节点的指针
    void *OSFlagNodeTCB;              //指向对应任务控制块的指针
    void *OSFlagNodeFlagGrp;          //反向指向信号量集的指针
    OS_FLAGS OSFlagNodeFlags;         //信号过滤器
    INT8U    OSFlagNodeWaitType;      //定义逻辑运算关系的数据
} OS_FLAG_NODE;
```

OSFlagNodeWaitType可选值及其意义

常数	信号有效状态	等待任务的就绪条件
WAIT_CLR_ALL 或 WAIT_CLR_AND	0	信号全部有效（全 0）
WAIT_CLR_ANY 或 WAIT_CLR_OR	0	信号有一个或一个以上有效（有 0）
WAIT_SET_ALL 或 WAIT_SET_AND	1	信号全部有效（全 1）
WAIT_SET_ANY 或 WAIT_SET_OR	1	信号有一个或一个以上有效（有 1）

表 48.1.2 OSFlagNodeWaitType 可选值及其意义

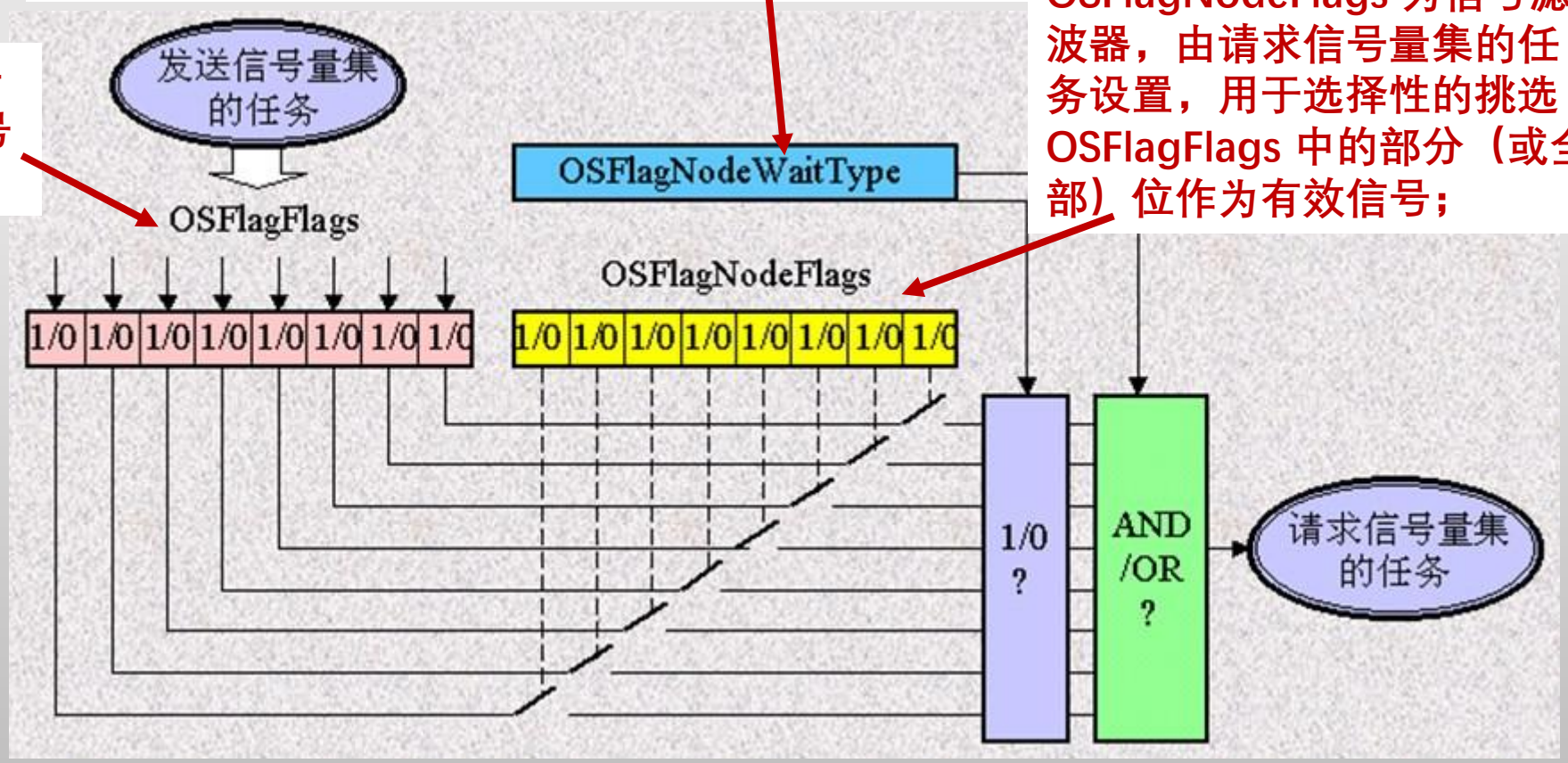
思考：假设请求信号量集的任务设置 OSFlagNodeFlags 的值为 0X0F，设置 OSFlagNodeWaitType 的值为 WAIT_SET_ANY，那么只要 OSFlagFlags 的低四位的任何一位为1，请求有效吗？如果低四位都为 0，请求有效吗？

标志组与等待任务共同完成信号量集的逻辑运算及控制：

OSFlagNodeWaitType 定义有效信号的逻辑运算关系，也是由请求信号量集的任务设置，用于选择有效信号的组合方式（0/1? 与/或？）。

OSFlagFlags 为信号量表，通过发送信号量集的任务设置

OSFlagNodeFlags 为信号滤波器，由请求信号量集的任务设置，用于选择性的挑选 OSFlagFlags 中的部分（或全部）位作为有效信号；



与信号量集相关的部分函数

1） 创建信号量集

OS_FLAG_GRP *OSFlagCreate (OS_FLAGS flags,INT8U *err);

flags 为信号量的初始值（即 OSFlagFlags 的值），

err 为错误信息

返回值为该信号量集的标志组的指针，应用程序根据这个指针对信号量集进行相应的操作。

2） 请求信号量集

OS_FLAGS OSFlagPend(OS_FLAG_GRP*pgrp, OS_FLAGS flags, INT8U wait_type, INT16U timeout, INT8U *err);

pgrp 为所请求的信号量集指针，

flags 为滤波器（即 OSFlagNodeFlags 的值），

wait_type 为逻辑运算类型（即 OSFlagNodeWaitType 的值），

timeout 为等待时限，

err 为错误信息

3）向信号量集发送信号：对信号量集标志组中的信号进行置“1”（置位）或置“0”（复位）的操作。

OS_FLAGS OSFlagPost (OS_FLAG_GRP *pgrp, OS_FLAGS flags, INT8U opt, INT8U *err);

pgrp：所请求的信号量集指针，

flags：选择所要发送的信号（指定对信号量集中的哪些信号进行操作）

opt：信号有效选项，opt =OS_FLAG_SET 为置“1”操作；

opt = OS_FLAG_CLR 为置“0”操作

err：错误信息

功能：所谓任务向信号量集发信号，就是对信号量集标志组中的信号进行置“1”（置位）或置“0”（复位）的操作。至于对信号量集中的哪些信号进行操作，用函数中的参数 flags来指定；对指定的信号是置“1”还是置“0”，用函数中的参数 opt 来指定（opt =OS_FLAG_SET 为置“1”操作；opt = OS_FLAG_CLR 为置“0”操作）。

20.8 UCOSII示例4-消息队列、信号量集和软件定时器（续）

3.软件定时器简介（ V2.83 版本后）

- 使得 UCOSII 的功能更加完善，在其上的应用程序开发与移植也更加方便。
- 在实时操作系统中一个好的软件定时器实现要求有较高的精度、较小的处理器开销，且占用较少的存储器资源。
- UCOSII 通过 OSTimTick 函数对时钟节拍进行加 1 操作，同时遍历任务控制块，以判断任务延时是否到时。
- 软件定时器时钟：同样由 OSTimTick 提供，但该时钟是在 UCOSII 的时钟节拍上面再做一次“分频”，软件定时器的最快时钟节拍就等于 UCOSII 的系统时钟节拍。这也决定了软件定时器的精度。

- 软件定时器的计数器： OSTmrTime
- 定时器管理任务 OSTmr_Task ：UCOSII 创建，高于应用程序中所有其他任务优先级，在这个任务中进行定时器的到时判断和处理。时钟节拍函数通过信号量给OSTmr_Task任务发信号。
- 软件定时器缩短了中断服务程序的执行时间，但也使得定时器到时处理函数的响应受到中断退出时恢复现场和任务切换的影响。
- 软件定时器功能实现代码存放在 tmr.c 文件中，移植时只需在 os_cfg.h 文件中使能定时器和设定定时器的相关参数。

- UCOSII 中软件定时器的实现方法: 将定时器按定时时间分组，使得每次时钟节拍到来时只对部分定时器进行比较操作，缩短了每次处理的时间。但这就需要动态地维护一个定时器组。定时器组的维护只是在每次定时器到时时才发生，而且定时器从组中移除和再插入操作不需要排序。这是一种比较高效的算法，减少了维护所需的操作时间。

- UCOSII 软件定时器实现了 3 类链表的维护：

①OS_EXT OS_TMR OSTmrTbl[OS_TMR_CFG_MAX]; //定时器控制块数组

软件定时器管理的基本单元，包含软件定时器的名称、定时时间、在链表中的位置、使用状态、使用方式，以及到时回调函数及其参数等基本信息

②OS_EXT OS_TMR *OSTmrFreeList; //空闲定时器控制块链表指针

空闲态的定时器控制块(OS_TMR)中，OSTmrnext 和 OSTmrPrev 两个指针分别指向空闲控制块的前一个和后一个，组织了空闲控制块双向链表。建立定时器时，从这个链表中搜索空闲定时器控制块。

- UCOSII 软件定时器实现了 3 类链表的维护：

③OS_EXT OS_TMR_WHEEL OSTmrWheelTbl[OS_TMR_CFG_WHEEL_SIZE];//定时器轮

该数组的每个元素都是已开启定时器的一个分组，元素中记录了指向该分组中第一个定时器控制块的指针，以及定时器控制块的个数。

运行态的定时器控制块(OS_TMR)中，OSTmrnext 和 OSTmrPrev 两个指针同样也组织了所在分组中定时器控制块的双向链表。

软件定时器主要函数

1） 创建软件定时器

OS_TMR *OSTmrCreate (INT32U dly, INT32U period, INT8U opt,
OS_TMR_CALLBACK callback,void *callback_arg, INT8U *pname, INT8U *perr);

dly，用于初始化定时时间，对单次定时（ONE-SHOT 模式）的软件定时器来说，这就是该定时器的定时时间，而对于周期定时（PERIODIC 模式）的软件定时器来说，这是该定时器第一次定时的时间，从第二次开始定时时间变为 period。

period，在周期定时（PERIODIC 模式），该值为软件定时器的周期溢出时间。

opt，用于设置软件定时器工作模式。可以设置的值为：OS_TMR_OPT_ONE_SHOT 或 OS_TMR_OPT_PERIODIC，如果设置为前者，说明是一个单次定时器；设置为后者则表示是周期定时器。

callback，为软件定时器的回调函数，当软件定时器的定时时间到达时，会调用该函数。

callback_arg，回调函数的参数。

pname，为软件定时器的名字。

perr，为错误信息。

软件定时器主要函数

2） 开启软件定时器

BOOLEAN OSTmrStart (OS_TMR *ptmr, INT8U *perr);

其中 ptmr 为要开启的软件定时器指针，

perr 为错误信息。

软件定时器主要函数

3） 停止软件定时器

任务可以通过调用函数 OSTmrStop 停止某个软件定时器，该函数的原型为：

OSTmrStop (OS_TMR *ptmr,INT8U opt,void *callback_arg,INT8U *perr);

其中 ptmr 为要停止的软件定时器指针。

opt 为停止选项，可以设置的值及其对应的意义为：

OS_TMR_OPT_NONE，直接停止，不做任何其他处理

OS_TMR_OPT_CALLBACK，停止，用初始化的参数执行一次回调函数

OS_TMR_OPT_CALLBACK_ARG，停止，用新的参数执行一次回调函数

callback_arg，新的回调函数参数。

perr，错误信息。

20.8 UCOSII示例4-消息队列、信号量集和软件定时器（续）

4.示例功能：创建 6 个任务：开始任务、LED 任务、触摸屏任务、队列消息显示任务、信号量集任务、按键扫描任务和主任务

开始任务：创建邮箱、消息队列、信号量集以及其他任务，之后挂起；

LED任务：控制LED0闪烁

队列消息显示任务：请求消息队列，在得到消息后显示到LCD；

信号量集任务：测试信号量集，采用 OS_FLAG_WAIT_SET_ANY 的方法，任何按键按下（包括 TPAD），该任务都会控制蜂鸣器发出“滴”的一声；

20.8 UCOSII示例4-消息队列、信号量集和软件定时器（续）

按键扫描任务：按键扫描，优先级最高，将得到的键值通过消息邮箱发送出去；

主任务：创建 1个软件定时器，(200ms 溢出一次，用于自动发送消息到消息队列)，并通过查询消息邮箱获得键值，根据键值执行 DS1 控制和软件定时器1的开关控制等。

os_cfg.h 里面修改软件定时器管理部分的宏定义：

#define OS_TMR_EN 1u //使能软件定时器功能

#define OS_TMR_CFG_MAX 16u //最大软件定时器个数

#define OS_TMR_CFG_NAME_EN 1u //使能软件定时器命名

#define OS_TMR_CFG_WHEEL_SIZE 8u //软件定时器轮大小

#define OS_TMR_CFG_TICKS_PER_SEC 100u //软件定时器的时钟节拍（10ms）

#define OS_TASK_TMR_PRIO 0u //软件定时器的优先级,设置为最高

这样我们就使能 UCOSII 的软件定时器功能了，并且设置最大软件定时器个数为 16，定时

main.c文件代码

任务设置

任务名称	任务优先级	任务堆栈大小	任务名称	任务优先级	任务堆栈大小
start_task	10	64	main_task	4	64
led_task	6	64	flags_task	3	64
qmsgshow_task	5	64	key_task	2	64

main.c文件代码

全局变量

```
OS_EVENT * msg_key; //按键邮箱事件块
OS_EVENT * q_msg; //消息队列
OS_TMR * tmr1;  //软件定时器 1
OS_FLAG_GRP * flags_key; //按键信号量集
void * MsgGrp[256];  //消息队列存储地址,最大支持 256 个消息
```

main.c文件代码　软件定时器 1 的回调函数

```
//每 100ms 执行一次,用于自动发送消息到消息队列
void tmr1_callback(OS_TMR *ptmr,void *p_arg)
{    u8* p,err;
     static u8 msg_cnt=0;  //msg 编号
     p=mymalloc(SRAMIN,13); //申请 13 个字节的内存
     if(p)
     {
          sprintf((char*)p,"ALIENTEK %03d",msg_cnt);
          msg_cnt++;
          err=OSQPost(q_msg,p); //发送队列
          if(err!=OS_ERR_NONE) //发送失败
          {
          myfree(SRAMIN,p);  //释放内存
          OSTmrStop(tmr1,OS_TMR_OPT_NONE,0,&err);  //关闭软件定时器 1
          }
     }
}
```

main.c文件代码 主函数

```
int main(void)
{
NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);//设置系统中断优先级分组 2
delay_init(168); //初始化延时函数
uart_init(115200); //初始化串口波特率为 115200
LED_Init();  //初始化 LED
LCD_Init();  //初始化 LCD
BEEP_Init(); //蜂鸣器初始化
KEY_Init();  //按键初始化
my_mem_init(SRAMIN);  //初始化内部内存池
OSInit(); //初始化 UCOSII
OSTaskCreate(start_task,(void *)0,(OS_STK *)&START_TASK_STK[START_STK_SIZE
-1],START_TASK_PRIO );//创建起始任务
OSStart();  //启动 UCOSII
}
```

main.c文件代码　开始任务

```
void start_task(void *pdata)
{
OS_CPU_SR cpu_sr=0;
u8 err;
pdata = pdata;
msg_key=OSMboxCreate((void*)0);  //创建消息邮箱
q_msg=OSQCreate(&MsgGrp[0],256); //创建消息队列
flags_key=OSFlagCreate(0,&err); //创建信号量集
OSStatInit(); //初始化统计任务.这里会延时 1 秒钟左右
OS_ENTER_CRITICAL(); //进入临界区(无法被中断打断)
//创建其他任务（略）
OSTaskSuspend(START_TASK_PRIO);  //挂起起始任务.
OS_EXIT_CRITICAL();  //退出临界区(可以被中断打断)
}
//
```

main.c文件代码　LED任务

```
void led_task(void *pdata)
{
    u8 t;
    while(1){
        t++; delay_ms(10);
        if(t==8)LED0=1; //LED0 灭
        if(t==100) {t=0;LED0=0;} //LED0 亮
    }
}
```

main.c文件代码　队列消息显示任务

```
void qmsgshow_task(void *pdata)
{
    u8 *p;u8 err;
    while(1)
    {
    p=OSQPend(q_msg,0,&err);//请求消息队列
    //显示消息：发送消息到PC机
    myfree(SRAMIN,p);
    delay_ms(500);
    }
}
```

main.c文件代码　主任务

```
//主任务初始化
void main_task(void *pdata)
{
u32 key=0;u8 err;
u8 tmr1sta=0; //软件定时器 1 开关状态
u8 flagsclrt=0; //信号量集显示清零倒计时
tmr1=OSTmrCreate(10,10,OS_TMR_OPT_PERIODIC,(OS_TMR_CALLBACK)
tmr1_callback,0,"tmr1",&err);  //100ms 执行一次
```

main.c文件代码　主任务主循环

```
    while(1)
    {     key=(u32)OSMboxPend(msg_key,10,&err);
        switch(key)
        {case 1: LED1=!LED1;break; //控制 DS1
        case 2: //控制软件定时器 1
        tmr1sta=!tmr1sta;
        if(tmr1sta)OSTmrStart(tmr1,&err);//启动软件定时器 1
        else OSTmrStop(tmr1,OS_TMR_OPT_NONE,0,&err);//关
        闭软件定时器 1
        break;}
        delay_ms(10);
    }
}
```

main.c文件代码　信号量集处理任务

```
void flags_task(void *pdata)
{
    u16 flags; u8 err;
    while(1)
    {
        flags=OSFlagPend(flags_key,0X001F,OS_FLAG_WAIT_SET_ANY,0,&err);//等待
        if(flags&0X0001)printf("KEY0 DOWN ");
        if(flags&0X0002) printf("KEY1 DOWN ");
        BEEP=1;
        delay_ms(50);
        BEEP=0;
        OSFlagPost(flags_key,0X001F,OS_FLAG_CLR,&err);//全部信号量清零
    }
}
```

main.c文件代码　按键扫描任务

```
void key_task(void *pdata)
{
    u8 key;
    while(1)
    {
        key=KEY_Scan(0);
        if(key)OSMboxPost(msg_key,(void*)key);//发送消息
        delay_ms(10);
    }
}
```